GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

KernelCAT：AI for Computing 实践分享

丁添
智子芯元联合创始人
深圳市大数据研究院 & 河套学院 副研究员



CONTENTS

Part 01 算力基建浪潮与AI Infra挑战

Part 02 案例1：AI+运筹优化，构建高精度数学函数算子

Part 03 案例2：RSI驱动的昇腾算子自动开发

Part 04 总结与展望



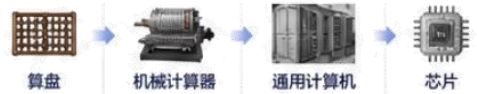
01

算力基建浪潮与AI Infra挑战



每一次计算能力的跃迁都代表着生产力的爆发

~ 1960s
计算工具化时代



- 天文学
- 航海测绘
- 金融学
- 早期工程设计

~ 2020s
计算规模化时代



- 航空航天
- 汽车工业
- 能源勘探
- 互联网与云计算
- 量化金融

2020s ~
计算智能化时代



- AI for Quantum
- AI for Science
- 自动驾驶
- 具身智能

从科学研究、工业生产到AI爆发，所有高价值产业都在被计算能力重新定义

AI产业爆发
- 算法激活Scaling Law
- 计算快、便宜且稳定

重塑工业生产
- 设计、模拟、优化
- 驱动工业智能化

突破科学边界
- 攻克复杂科学问题
- 推动技术范式革新

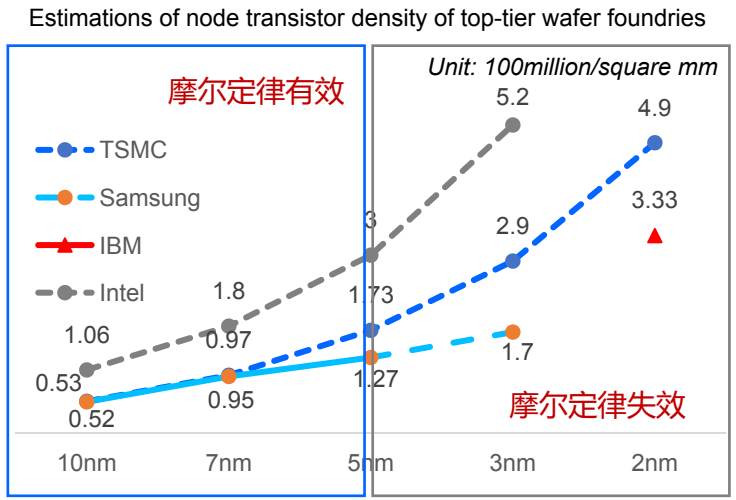
摩尔定律放缓，亟需算法补位

全球 AI 算力需求高速增长，硬件制程进入物理极限

全球 AI 计算总量（FP16）同比增长 4,000倍



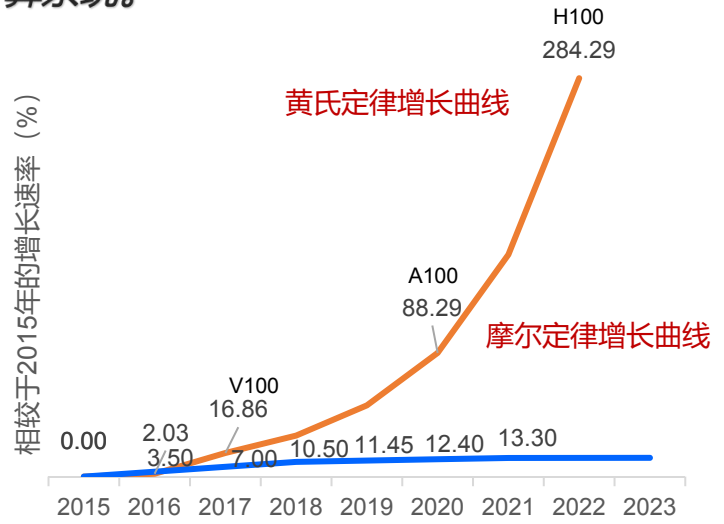
摩尔定律在 5nm 后开始放缓 硬件制程进入物理极限



算法效率和底层实现效率突破，引领实际计算性能提升

*“我们创立公司，是为了**加速软件**，芯片设计反而是次要的。英伟达之所以能快速创新，正是因为我们从**底层算法**出发，全面掌握计算系统。”*

——黄仁勋



实际计算性能

=

理论峰值计算性能

✖

算法效率

✖

硬件资源调度效率

算力供需失衡被快速拉大

综合算力增长依赖软硬件深度协同优化

数据来源：华为《计算2030》研究报告，https://www-file.huawei.com/dam/asset/view/d459b1ffbafc4be18ff0ce4a77d16bfb.pdf

高效计算方案的供需失衡

供给：买到芯片不等于拥有可用算力

用起来难，需克服软硬件层的复杂度

主流 AI 资产长期围绕 CUDA 等成熟生态沉淀，**不同模型、硬件与场景的最优实现路径高度分化**，需要大量跨模型、算子、编译、调度的专家经验

从可用到实现极限性能空间巨大

FlashAttention V4 版相比 PyTorch 基准提速 **20 倍**
DeepGEMM 相比 CUDA 21.8 速度提升 **2.7 倍**
根据任务场景做框架优化相比官方极限提升 **5 倍**

人工开发门槛高耗时长

软件 / 硬件 / 算法 / 运筹多学科综合问题
优化目标多 且相互冲突，**参数空间大** 需反复试错，调试与验证成本极高

AI for Computing
优化计算是必然

需求：Workload井喷，复杂度指数增长

软硬件架构加速更迭

软件侧：新架构层出不穷
硬件侧：GPU / NPU / TPU 每年 1-2 代更新；存算一体，可重构计算，Chiplet 等不断创新

特化程度加深，垂直场景需求涌现

从 Chat 时代到 **Agent 时代** Workload 井喷
自动驾驶 / 具身智能 / 科学计算 / 端侧场景等都需要 **场景化高效计算**

计算复杂度超越人类处理极限

模型结构、算子实现、编译策略、内存布局、通信拓扑与硬件微结构相互牵制，传统专家基于计算架构分层局部调整，**难以端到端全局优化**

算法自动发现

将计算任务建模为算法空间，在精度和效率的约束下生成新算法

筛选候选算法，牵引算法空间内的高效探索

发明人类从未发现的新算法、新计算路径

AI for Computing

提供约束与验证环境

任务-资源协同建模优化

将计算任务转化为约束优化问题，大模型负责任务理解和代码生成，运筹优化负责自动建模和寻优

搜索切分 / 访存 / 并行 / 调度最优策略

更高效、更低成本的硬件特化实现

硬件在环的闭环验证

编译 → 运行 → Profiling → 精度 / 性能评估 → 反馈迭代

算法效率 × 理论峰值计算性能 × 硬件资源调度效率

= 实际计算性能

02

AI+运筹优化，构建高精度数学函数算子



问题背景：数学函数的最优近似

- 复杂的代数函数、超越函数在科学计算中普遍存在
  - exp，sqrt，sin，cos，…
  - 神经网络数学算子：softmax，sigmoid，Swish，…
- 实际计算系统中，需要用数学手段近似
  - e.g.，四则运算组合（+，-，×，÷）的有理式
- **如何针对特定系统寻找最佳的近似手段？**
  - ➢ 传统：人工推导形式+系数（泰勒级数、Padé近似）
  - ➢ 低精度下（FP32，FP16，FP8……）传统方法失效
  - ➢ DeepMind：Auto-Numerics-Zero（ANZ）
    - ➢ “自动搜索”取代“人工设计”

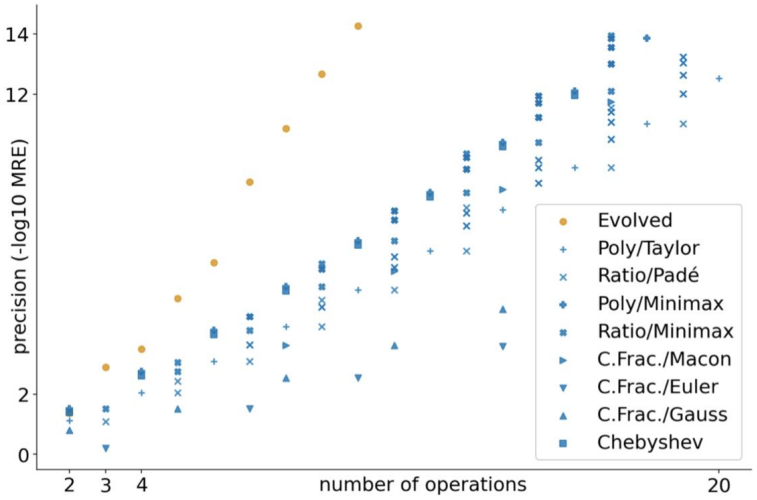
案例：exp函数的有理式近似

Padé近似

$$R_{4,4}(r) = \frac{1 + \frac{1}{2}r + \frac{1}{10}r^2 + \frac{1}{120}r^3 + \frac{1}{5040}r^4}{1 - \frac{1}{2}r + \frac{1}{10}r^2 - \frac{1}{120}r^3 + \frac{1}{5040}r^4}$$

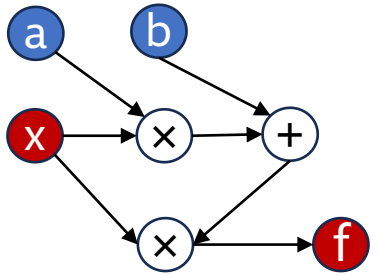
$$r \in [-0.5 \ln 2, +0.5 \ln 2]$$

DeepMind: Auto-Numerics-Zero (ANZ)



搜索空间

- **有理式的编码：有向无环图（DAG）**
  - 节点：输入/输出节点，系数节点，运算节点
  - 有向边：定义内部输入输出关系
  - 系数：有限精度（FP32，FP64，…）



$$f(x;a,b) = (ax+b)x$$

约束与优化目标

- 对于给定的目标函数 g(x)，定义最大相对误差

$$\mathrm{MRE}(f) = \max_{x\in T}\left|\frac{f(x)-g(x)}{g(x)}\right|$$

$$\min_{f\in\mathcal{F}_{\mathrm{DAG}}} L(f) \quad \text{s.t.} \quad \mathrm{MRE}(f;g) \le \epsilon$$

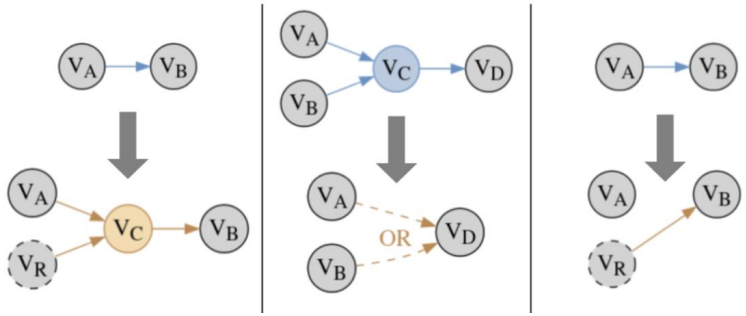
- 有理式的“复杂度” $L(f)$：实际运行时间/运算数量/……

搜索空间不连续、难以参数化，目标函数非凸

总体搜索流程

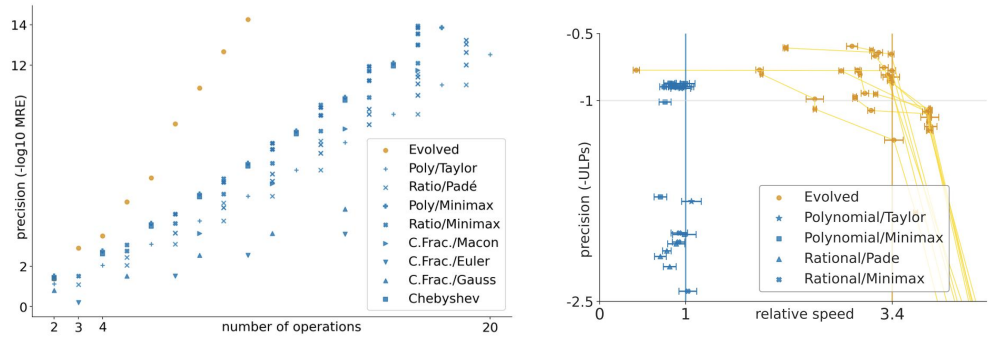


生成新DAG拓扑



增加节点　　删除节点　　重连节点

多目标帕累托评估

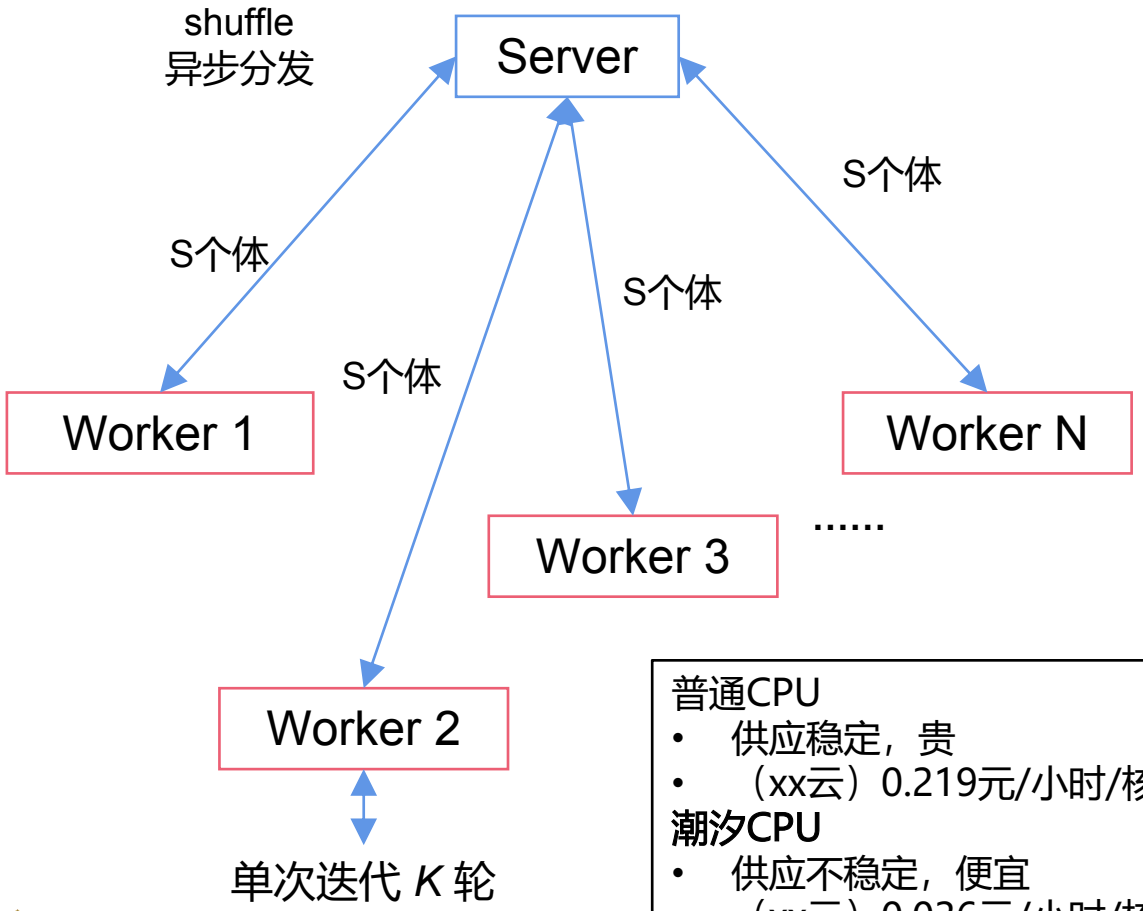


- 根据效率-精度帕累托面筛选

挑战在哪里？

- **外循环**：一个实验迭代1,000,000,000（10亿）DAG
  - 效率低、成本高
  - 单核CPU优化一个DAG系数：~30秒
  - 300亿秒 = **951年**！
- **内循环**：使用CMA-ES优化DAG系数
  - 非常成熟的算法，**无法复现论文精度……**
- **结果**：只给出了对exp函数在[0,1]区间的拟合结果
  - 数学函数算子需要针对不同分支逐区间拟合
  - **搜索效率和精度亟需量级提升！**

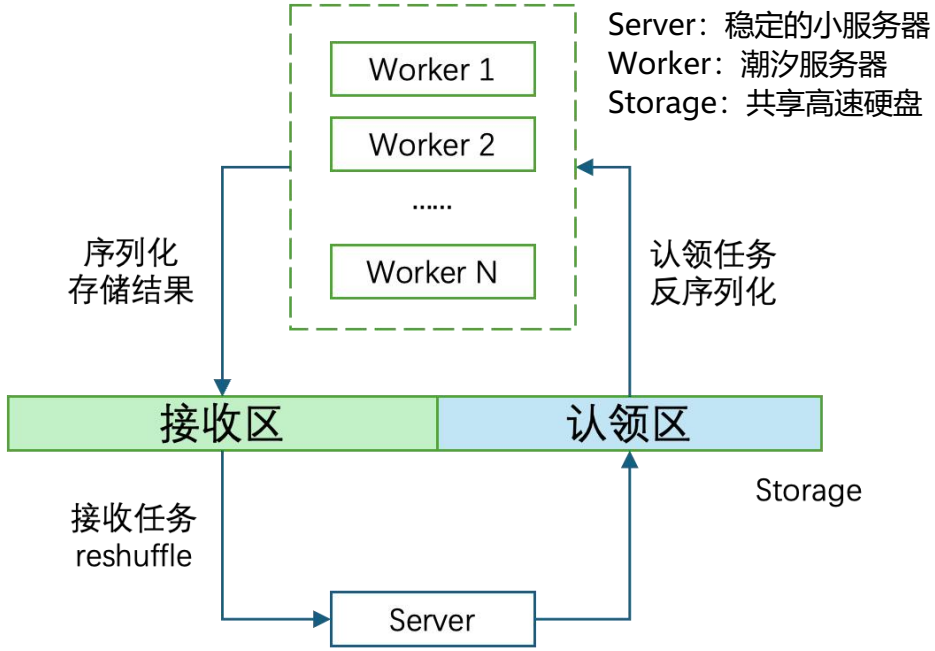
外循环：基于潮汐算力与异步通信的大规模搜索



普通CPU
- 供应稳定，贵
- （xx云）0.219元/小时/核

潮汐CPU
- 供应不稳定，便宜
- （xx云）0.036元/小时/核

已支持千核级别分布式搜索



- **实现技术**：**基于高速硬盘的任务认领机制**
  - Server将任务打包为镜像
  - Worker从硬盘直接拉起服务
  - 任务完成，Worker将结果存储到硬盘

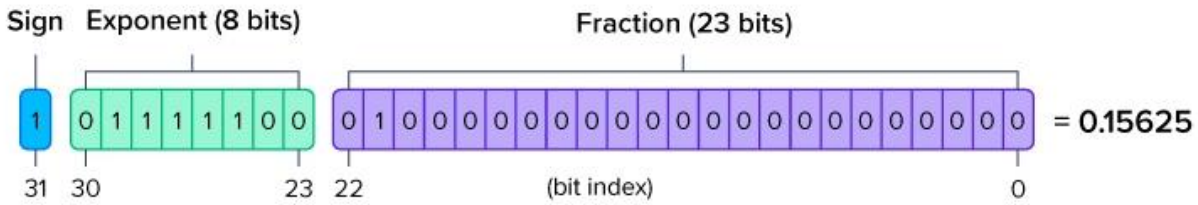
- 实现**动态征用和释放worker**

➢ 充分利用潮汐服务器

内循环：DAG系数拟合问题Landscape

- **大尺度连续**，“看似光滑”
- 高度非凸
  - 多峰，大量局部最优
  - sharp minima → 传统CMA-ES容易失效

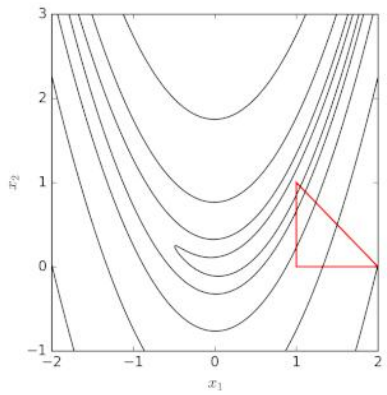
- **小尺度离散**
  - FP32空间本质是离散点
  - 类似平台函数



内循环：2-Stage系数优化策略

- **大尺度策略**：搜索全局较优区域（”good basin”）
  - 充分利用连续性下的局部曲率信息”
  - Levenberg－Marquardt（LM）算法
- **小尺度算法**：搜索区域内最优解（basin底部）
  - 无需求导，在局部的“破碎”区域做邻域搜索
  - Nelder-Mead算法

大尺度策略 – 其他连续优化	
SGD，Adam	收敛点震荡
GD，牛顿法	局部最优点，精度不足



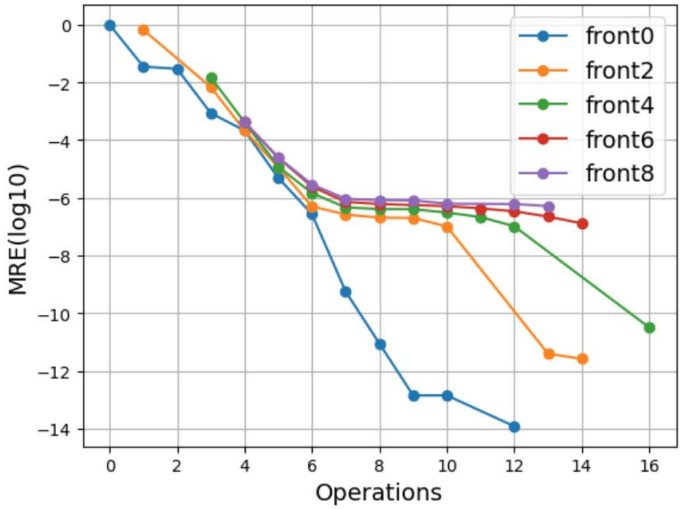
小尺度策略 – Nelder-Mead算法示意

目标函数（DAG拓扑）	方法	误差（MRE）（FP64）	时间（s）
F6	CMA-ES	2.8e-7	0.46
	CMA-ES + LM	6.25e-7	1.56
	LM	6.25e-7	1.13
	2-Stage	2.8e-7	1.14
F8	CMA-ES	8.7e-6	3.72
	CMA-ES + LM	2.07e-11	4.97
	LM	2.08e-11	6.59
	2-Stage	9.56e-12	6.60
F10	CMA-ES	3.05e-6	4.09
	CMA-ES + LM	7.1e-15	6.31
	LM	5.44e-15	2.34
	2-Stage	4.21e-15	2.35

exp函数：拟合范围[0,1]，迭代 2160万 DAG

操作数	Ours MRE精度	Pade逼近 MRE精度
0	0.999006929070095	-
1	0.035464	0.10363832351432703
2	0.02976203760147518 7	-
3	0.000865	0.10363832351432703
4	0.000216	0.0803013970713941 5
5	4.955150811673234 5e-06	-
6	2.730128247130411 3e-07	-
7	5.624346905908829e-10	0.0014700882489423336
8	8.928194012762046e-12	-
9	1.423872125158078 8e-13	-
10	1.423305917569450 7e-13	0.0001225445083874411
12	1.208517903606661 6e-14	-

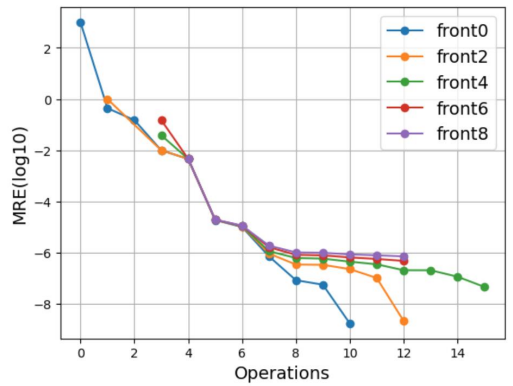
```
def f8(x0):
    C0 = 0.9999999999953635
    C1 = 125.31792833352682
    C2 = 62.658963057944355
    C3 = 947.1234855012225
    C4 = -361.59110729835595
    op0 = C4 / x0
    op1 = op0 - x0
    op2 = C3 / op1
    op3 = op1 + op2
    op4 = op3 + C2
    op5 = C1 / op4
    op6 = C0 - op5
    op7 = op6 * op6
    return  op7
```



log函数：拟合范围[1,2]，迭代 640万 DAG

操作数	Ours MRE精度	Pade逼近 MRE精度
0	1000.5004166251254	-
1	0.4420571448893068	0.4426950408896344
2	0.1505502084671 4712	-
3	0.009936906491473604	0.03820330607402442
4	0.004584892687863 8155	-
5	1.879498882105572e-05	-
6	1.0015722977566789e-05	0.009886528622274356
7	7.210291224824698e-07	0.0012111253840994
8	8.388801628111407e-08	-
9	5.612456333444749e-08	-
10	1.7206724158826312e-09	-

```
def f10(x0):
    C0 = 63.76154131434437
    C1 = 90.83490998375169
    C2 = 77.48051336255907
    C3 = 77.17018906690362
    C4 = 154.59645195416158
    C5 = 0.9999999999943332
    C6 = -3389.46503798873
    op0 = C5 - x0
    op1 = C4 / op0
    op2 = C2 - op1
    op3 = C0 / op2
    op4 = op2 + op0
    op5 = C6 / op4
    op6 = op1 - op5
    op7 = C3 - op6
    op8 = C1 / op7
    op9 = op3 + op8
    return  op9
```

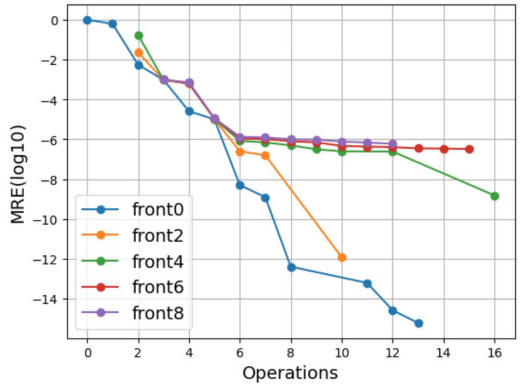


新方法表现：sigmoid

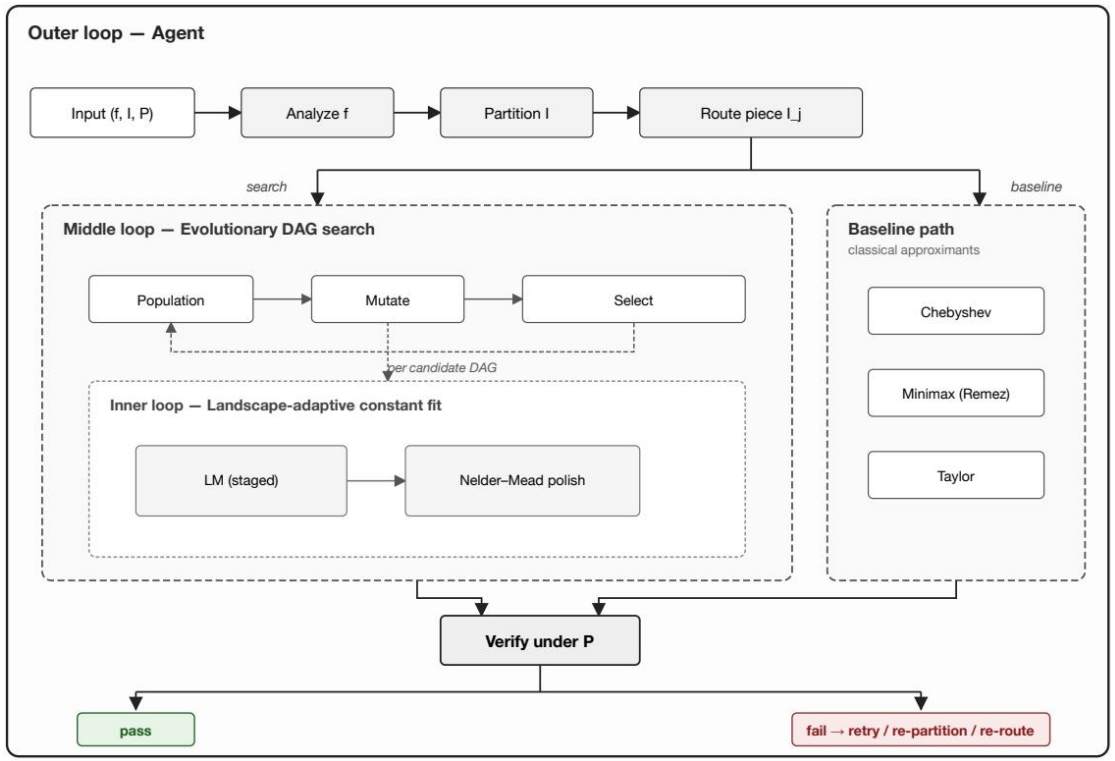
sigmoid函数：拟合范围[0,1]，迭代 640万 DAG

操作数	Ours MRE精度	Pade逼近 MRE精度
0	0.998000999500166	
1	0.6238825834428109	1.0518191617571635
2	0.005602878667776907	
3	0.0009659765462539 85	
4	2.577024529185390 2e-05	
5	9.92889283142378e-06	
6	5.00321058749996 1e-09	0.772020093138093
7	1.2470960194009668e-09	0.4298455666312029
8	4.044168280081307e-13	
11	6.180910589534705e-14	
12	2.6214160995833207e-15	
13	6.079513499775499e-16	

```
def f12(x0):
    C0 = -0.5000000000000012
    C1 = 0.004605639023286253
    C2 = 7.168133229373288
    C3 = 1.1238233782015976
    C4 = 391.0212395059102
    C5 = 89.06430566040987
    op0 = C5 / x0
    op1 = x0 + op0
    op2 = C4 / op1
    op3 = op2 + x0
    op4 = C3 * op3
    op5 = C1 * op4
    op6 = C4 / op4
    op7 = op6 + op3
    op8 = C2 / op7
    op9 = op5 + op8
    op10 = op8 - C0
    op11 = op9 + op10
    return  op11
```



- **构建算法自动发现Agent**
  - 引入LLM，自动处理任务分解、函数分区
  - 将传统数学方案引入搜索域



Task	Ours: DAG ops by interval	Expanded baseline ops	Ours smaller by
sigmoid fp16	$[0,.5]:5,\ [.5,1]:7,\ [1,2]:4,$ $[2,3]:9,\ [3,4]:8,\ [4,8]:7$	≈ 12–20	$\geq \mathbf{1.3}\times$
tanh fp16	$[0,.25]:3,\ [.25,1]:6,\ [1,5]:9$	≈ 19–33	$\geq \mathbf{2.1}\times$
sinh fp16, low/mid	$[0,3]:10,\ [3,5]:11,\ [5,7]:7,$ $[7,9]:8$	≈ 20–35	$\geq \mathbf{1.8}\times$
sinh fp16, high	$[9,9.907]:6,\ [9.907,10.375]:$ $9,\ [10.375,11]:11,\ [11,11.75]:$ 10	≈ 16–20	$\geq \mathbf{1.4}\times$
tanh fp32	$[0,.3]:6,\ [.3,.5]:7,\ [.5,.7]:7$	≈ 19–32	$\geq \mathbf{2.7}\times$

常规函数：超越传统数学库baseline

- 实现多种精度数学函数算子的全自动搜索
- 帮助昇腾算子库补缺补弱
  - lgamma，Bessel等算子**精度提升3个量级**
    - 精度齐平CUDA
    - 落入昇腾950数学库
- **计算数学专家1个月工作量 ➜ 3天搜索用时**

Abbasov et al., "*Precision-Aware Automated Discovery of Numerical Function Approximations via Landscape-Adaptive Search and LLM Agents*", NeurIPS 2025 AI4Math Workshop

03

RSI驱动的昇腾算子自动开发



同一算法存在多种计算实现

算子是 GPU / NPU 上
执行具体计算的底层程序

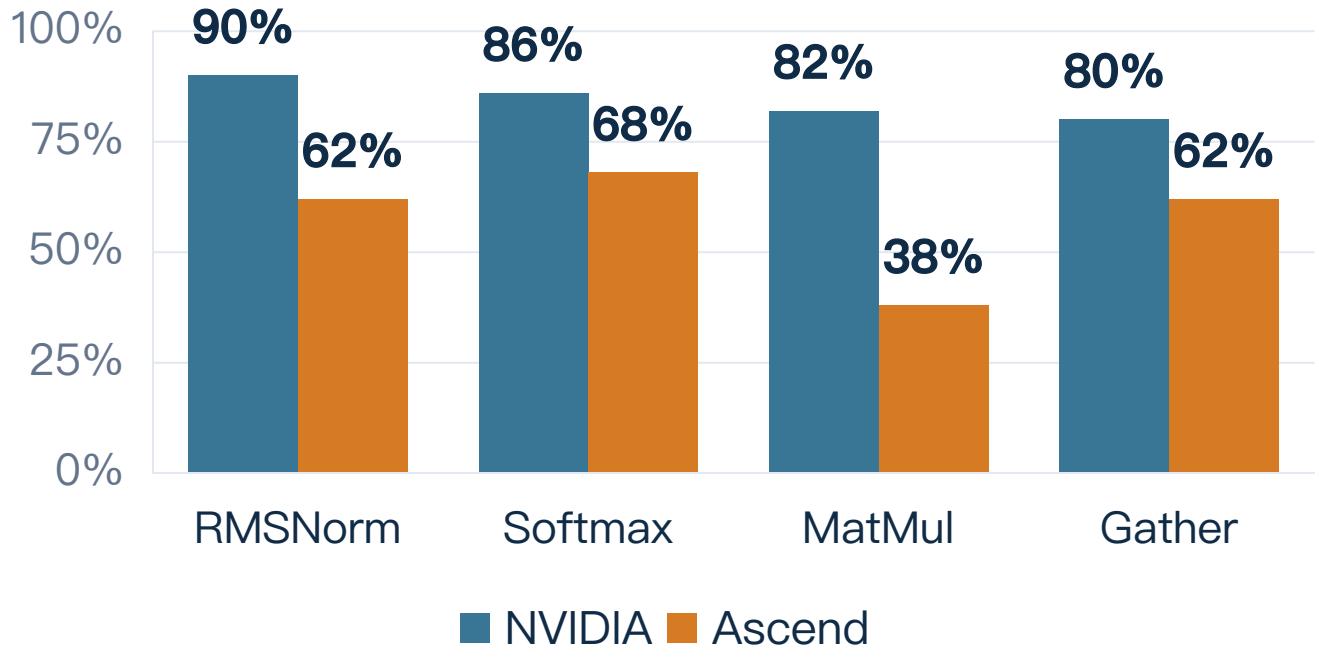
正确

满足输入、布局和精度约束

效率

选择分块、访存和缓冲策略

先导实验：生成代码正确率



任务与约束优化空间

任务约束　　在固定评测规则与共享预算下，搜索最优内核实现

任务定义与评测规则 → 共享预算 B → 通过验证的最快程序

可优化的实现

算法	计算结构	访存方式	并行划分	执行调度	硬件映射
改变计算量	分解与融合	数据复用与布局	核间任务分配	流水重叠与同步	指令与资源配置

➤在任务规则与预算 B 内，优化算法、计算结构、访存、并行划分、流水调度和硬件映射。

- 如何让Agent在长程探索中持续优化算子性能
- 如何让探索经验跨任务泛化？


① Context
持久知识库 K
来自原始资料和历史执行记录
工作上下文 C_t
② Search / Agent Harness
线程内生成代码
编译与设备评测
执行轨迹
Controller
选择父代、分配提案预算、调度线程
修正优化假设
③ Replay

关键设计：轨迹复盘与知识修订

变异前的提案

父代代码与知识条目版本
修改方案与预期效果
新增开销与停止条件

Replay

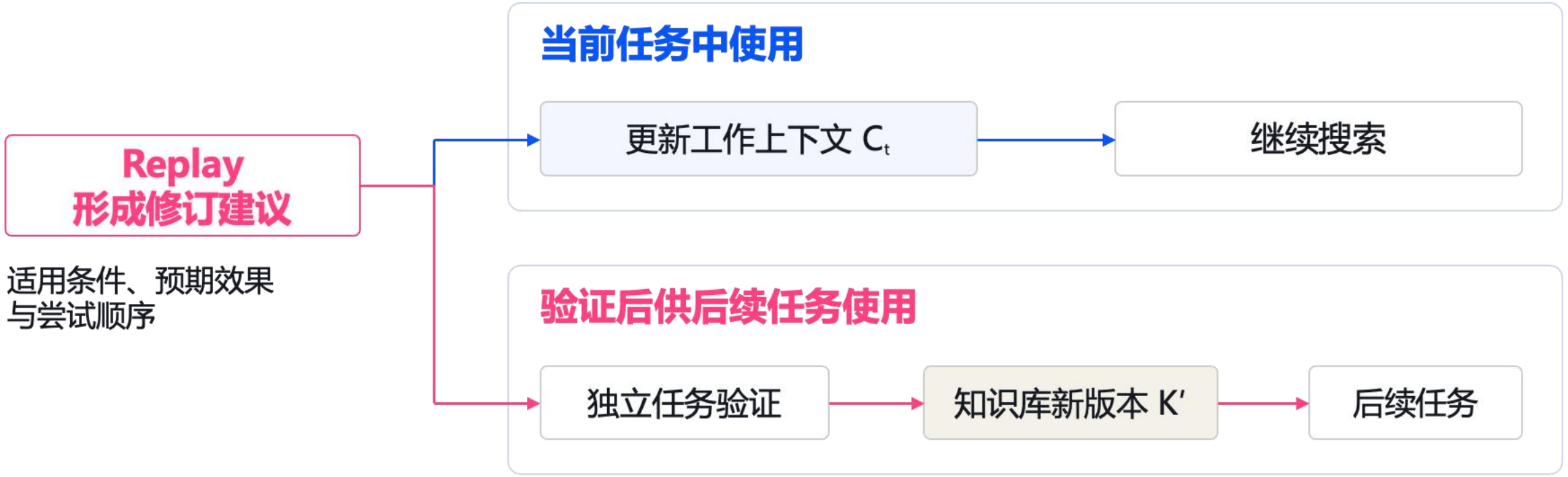
设备评测回执

实际代码与正确性结果
相同条件下的父代、子代性能
性能分析结果与资源开销

代码未按提案实现	适用条件不匹配	证据不足
修正代码，再次验证	补充条件，调整修改方案	暂不下结论，补充验证

➤Replay 对照预期与执行反馈，区分实现偏差、条件失配和证据不足，修正下一轮优化假设。

工作上下文修订与知识库更新

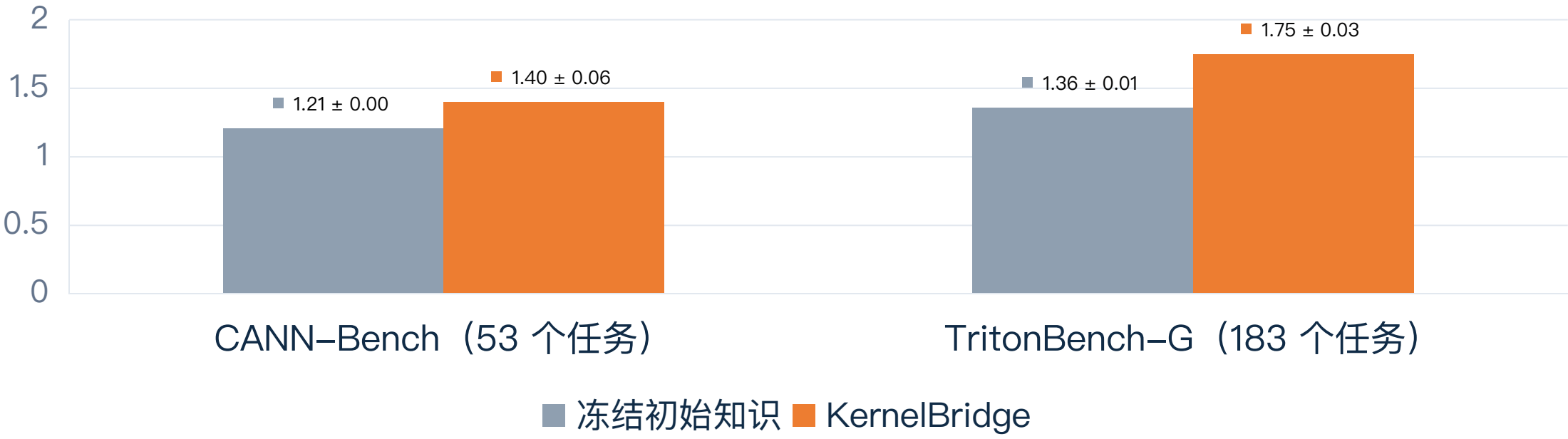


➤任务内先修订工作上下文，指导后续搜索；可复用规则经独立任务验证后，再写入持久知识库。

知识更新提高最终平均加速比

同一框架、搜索与评测规则，仅知识更新逻辑不同

- 使用5种模型，每任务 40 次编译，3 次重复实验下最佳平均加速比



获得CANNBench A3系列榜首成绩

CANNBench 概览 评测集 排行榜 提交 文档

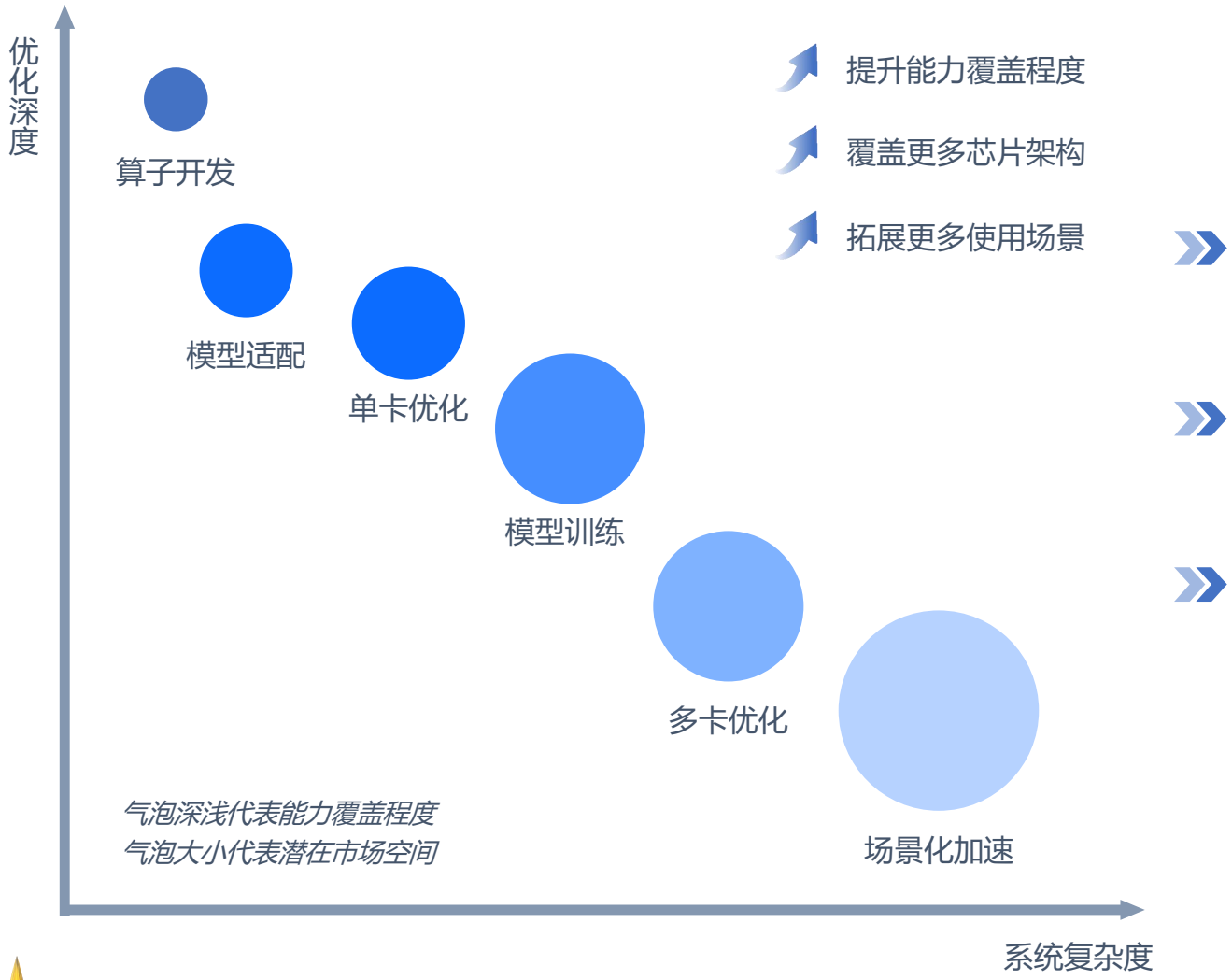
最高总分	中位基准加速比	通过率	提交算子
8099.48	0.749×	85.6%	53/53
SLAI-CUHKSZ-KernelCAT · A3	理论 0.07× · 基于 162 个方案	147449 / 172240 测例	完整任务集

#	方案	硬件	用户	总分 ↓	平均分 ↕	基准加速比 ↕	理论加速比 ↕	提交算子 ↕	测例 ↕	更新 ↕
01	SLAI-CUHKSZ-KernelCAT 106 任务 · official-tasks · 1.1	A3	m0_73877465	8099.48	76.41	1.362×	0.107×	53/53	99.5%	20
02	SwissTeam 106 任务 · official-tasks · 1.1	A3	hpccsg Huawei Techno…	8097.33	76.39	1.351×	0.108×	53/53	98.8%	20
03	infynai-kernaut 98 任务 · official-tasks · 1.1	A3	nebuladream 独立开发者	8032.19	75.78	1.335×	0.099×	53/53	99.1%	20
04	CANNBOT LINGXI-EVO 106 任务 · official-tasks · 1.1	A3	yyzhang0715	8004.19	75.51	1.220×	0.101×	53/53	98.9%	20
05	Debug-Agent Claude Code Kimi K3 106 任务 · official-tasks · 1.1	A3	GitCodeVoya…	7737.05	72.99	0.942×	0.072×	53/53	99.5%	20
06	NextAISystems 103 任务 · official-tasks · 1.1	A3	NextAISyste…	7456.24	70.34	0.897×	0.073×	53/53	98.1%	20
07	CANN-EvoAgent-deepseekv4flash0731 72 任务 · official-tasks · 1.1	A3	jiaqian	7409.73	69.90	0.902×	0.067×	53/53	95.1%	20
08	EvoScientist	A3	Evoscientis…	7192.67	67.86	0.662×	0.043×	53/53	98.1%	20

04

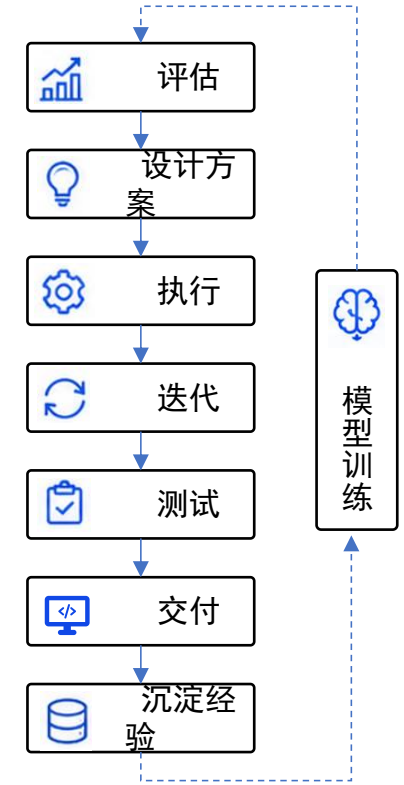
总结与展望





数据飞轮

基于 Agent 轨迹数据训练模型



端到端闭环 + 持续提升

任务级自动求解
更快定位与更优解

持续自我优化
越用越聪明

经验复用
规模化交付

已服务1000+工程师与硕博生，
收集真实国产芯片适配任务agent数据200多万条

驱动算力系统跨层重构



多层次递归自改进



GOSIM

GOSIM Shenzhen 2026

Thanks!

